

AMBIENTE DI PROGRAMMAZIONE (IDE=INTEGRATED DEVELOPMENT ENVIRONMENT)

- **Editing**: scrittura del codice sorgente (.java)
- **Compilazione**: traduzione del codice sorgente in codice eseguibile
- **Esecuzione**: traduzione del codice eseguibile in codice macchina

COMPILAZIONE ED ESECUZIONE: dipendenza da ambiente hardware (processore) e software (sistema operativo).

Il linguaggio Java per la compilazione dei programmi utilizza un altro approccio. La compilazione produce file scritti in un particolare linguaggio detto BYTECODE.

BYTECODE = linguaggio più astratto (alto livello) rispetto al linguaggio macchina.

Pertanto, il bytecode è indipendente dall'ambiente di esecuzione.

Il bytecode, per poter essere eseguito, necessita di un software chiamato JAVA VIRTUAL MACHINE.

Ogni piattaforma ha la sua JVM che **interpreta** il codice compilato e lo esegue su una specifica macchina reale.

PIATTAFORMA: per piattaforma si intende sia l'architettura hardware (processore) sia l'architettura software (SO) installata sulla macchina.

PORTABILITA': IL JAVA E' UN LINGUAGGIO PORTABILE.

Portabilità: capacità di un programma di essere eseguito su piattaforme diverse senza dover essere modificato e ricompilato.

PROGRAMMARE IN JAVA

Per programmare in Java è sufficiente:

- Un editor di testo (blocco note di Windows)
- JDK (JAVA DEVELOPMENT KIT)

NOTA BENE: il JDK include la JVM

Il JDK contiene:

- **JAVAC:** COMPILATORE CHE TRASFORMA IL CODICE SORGENTE IN BYTECODE
- **JDB:** SOFTWARE PER IL DEBUGGING DEL CODICE
- **API** (APPLICATION PROGRAMMING INTERFACE): LIBRERIE DEL LINGUAGGIO JAVA
- **JRE** (JAVA RUNTIME ENVIRONMENT): AMBIENTE DI ESECUZIONE DEI PROGRAMMI SCRITTI IN JAVA. A QUESTO AMBIENTE APPARTIENE LA JVM

JAVA VIRTUAL MACHINE = SOFTWARE/PROGRAMMA

JVM = COMPUTER ASTRATTO -> SIMULA BUONA PARTE DELLE FUNZIONALITA' DI UN COMPUTER REALE.

QUESTA **ASTRAZIONE** OFFRE LA POSSIBILTA' DI SCRIVERE PROGRAMMI PORTABILI IN MODO CHE POSSANO ESSERE ESEGUITI IN AMBIENTI OPERATIVI DIVERSI E CON DIFFERENTI CARATTERISTICHE HARDWARE.

JVM ESEGUE IL BYTECODE E LE LIBRERIE DI BASE DEL JAVA.

TRADUZIONE DEL CODICE SORGENTE: APPROCCIO JAVA

Per tradurre il codice sorgente in codice binario esistono tre approcci:

- **COMPILATO**
- **INTERPRETATO**
- **APPROCCIO JAVA**

APPROCCIO COMPILATO

FILE SORGENTE (.c o .cpp) → COMPILATORE → FILE OGGETTO (.obj) → LINKER (collega le librerie) → PROGRAMMA ESEGUIBILE (.exe)

FILE OGGETTO è scritto in codice binario.

FILE ESEGUIBILE è scritto in codice binario.

Quando l'utente vuole eseguire il programma, lo avvia e le istruzioni, scritte in linguaggio macchina, vengono caricate nella RAM.

ESEMPIO:

se si vuole scrivere un programma funzionante su un sistema operativo Windows, si dovrà scrivere il codice sorgente utilizzando le librerie specifiche del particolare sistema operativo. IL CODICE SORGENTE SARÀ TRADOTTO IN BINARIO MEDIANTE L'UTILIZZO DELL'APPOSITO COMPILATORE CHE TRADUCE PER IL SO WINDOWS A 64 BIT.

Pertanto, il programma eseguibile che si ottiene è specifico per una certa piattaforma.

APPROCCIO INTERPRETATO

Nei linguaggi interpretati (JavaScript, PHP) la fase di compilazione e di esecuzione avvengono contemporaneamente.

INTERPRETE ≠ COMPILATORE

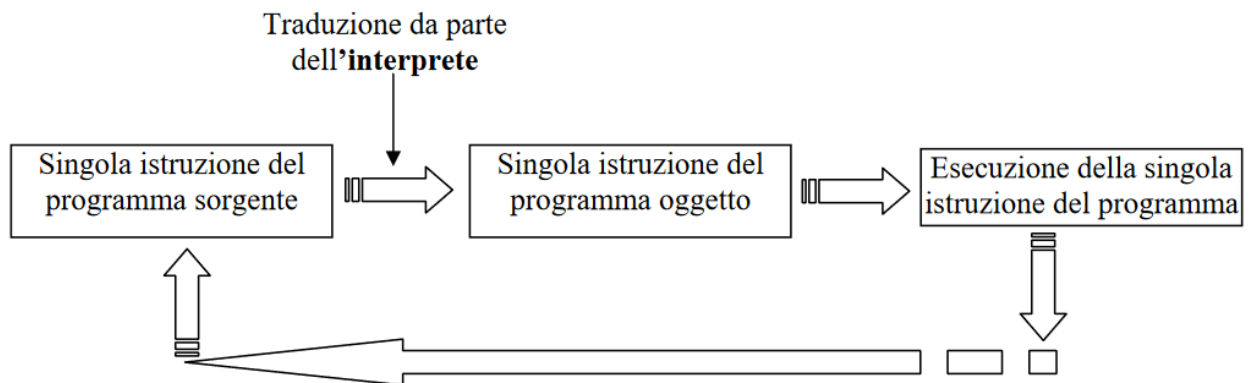
INTERPRETE E COMPILATORE SONO SOFTWARE.

L'interprete è un software che traduce riga per riga il codice sorgente in binario a tempo di esecuzione (run-time). **L'istruzione tradotta viene immediatamente caricata nella RAM ed eseguita.**

L'approccio interpretato comporta una minore efficienza a run-time: un programma interpretato richiede più memoria ed è meno veloce a causa dell'OVERHEAD introdotto dall'interprete stesso.

OVERHEAD: RISORSE ACCESSORIE, RICHIESTE IN SOVRAPPIÙ RISPETTO A QUELLE STRETTAMENTE NECESSARIE PER OTTENERE UN DETERMINATO OUTPUT.

Durante l'esecuzione, infatti, l'interprete deve analizzare le istruzioni a partire dal livello sintattico, identificare le azioni da svolgere (eventualmente trasformando i nomi simbolici delle variabili nei relativi indirizzi di memoria), ed eseguirle.

Schema della fase di **interpretazione** di un programma sorgente

*L'interprete traduce la prima istruzione del programma e la esegue;
 L'interprete traduce la seconda istruzione del programma e la esegue;

 L'interprete traduce l'ultima istruzione del programma e la esegue.*

Compilazione

1. si traducono tutte le istruzioni del programma
2. si eseguono in ordine le istruzioni ottenute

Interpretazione

1. si traduce una singola istruzione alla volta (di alto livello)
2. si esegue l'istruzione ottenuta
3. si passa all'istruzione successiva

COMPILAZIONE VS INTERPRETAZIONE

Compilazione: maggiore efficienza e sicurezza per quel che riguarda la rilevazione degli errori, in quanto questi ultimi vengono rilevati prima che il codice sia eseguito.

Interpretazione: poiché ciascuna istruzione viene tradotta e poi eseguita, se ad esempio si rileva un errore alla riga 100, la macchina dovrà aver bisogno di tradurre ed eseguire le 99 righe precedenti prima di rendersi conto dell'errore.

COS'È L'INTERPRETE?

L'interprete è un software più lento rispetto al compilatore perché nel caso del compilatore viene prodotto un eseguibile che occorrerà lanciare (eseguire); nel caso dell'interprete, occorrerà attraversare ogni volta la fase di TRADUZIONE prima dell'esecuzione. IL PROGRAMMA DEVE ESSERE TRADOTTO OGNI VOLTA AUMENTANDO I TEMPI DI ESECUZIONE.

- L'esecuzione di un programma richiede che l'interprete sia presente in memoria centrale.
- Il compilatore non necessita di essere caricato in memoria centrale.
- Il codice eseguibile (.exe) prodotto dalla compilazione può essere eseguito solo in determinate macchine e sistemi operativi: CODICE NON PORTABILE. (COMPILATORE)
- Gli errori di compilazione impediscono al programma di essere eseguito (COMPILATORE).
- Nell'approccio interpretato gli errori sono visibili solo se il programma viene mandato in esecuzione.

APPROCCIO JAVA

Il linguaggio Java è sia compilato sia interpretato.

1. Codice sorgente tradotto dal compilatore Java in BYTECODE (.class). Quest'ultimo non è il codice binario eseguito. **A differenza dei linguaggi compilati, il bytecode è uguale per tutte le piattaforme.**
2. Il bytecode deve essere eseguito mediante la Java Virtual Machine.
NEL CASO DEL LINGUAGGIO JAVA, LA JVM RAPPRESENTA L'INTERPRETE.
3. La JVM prenderà il bytecode e lo tradurrà in binario affinché il processore possa eseguire le singole istruzioni.
MAGGIORE PORTABILITÀ: IL BYTECODE PUO' ESSERE DISTRIBUITO A TUTTE LE PIATTAFORMA.

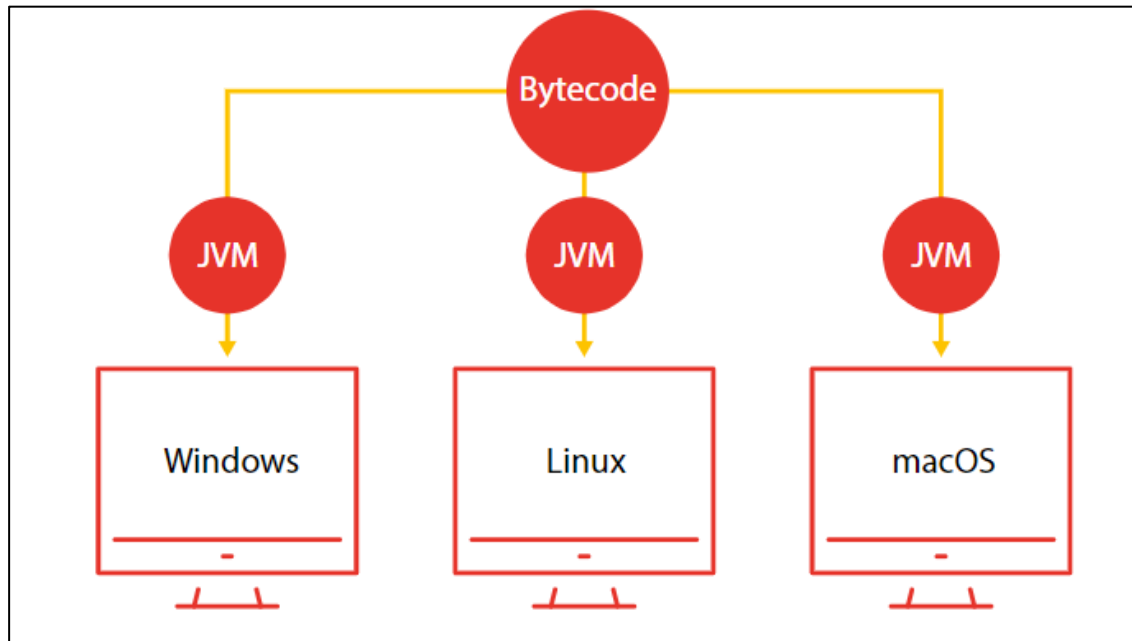


Figura 1: bytecode e Java Virtual Machine.

NOTA: vedi paragrafo 1 del libro di testo *Corso di Informatica 2 - Zanichelli* (pagina A28 e A29)